

AWS + CLOUD COMPUTING

Fundamentals for Client Interviews

Concept-only study guide | Beginner to interview-ready fundamentals

Designed for this week

This guide assumes you are starting cloud computing and AWS fundamentals from zero. It deliberately prioritizes concepts, architecture, vocabulary, and interview explanations over hands-on labs. You do not need to memorize every AWS service.

What this guide gives you

- A clean mental model of cloud computing before AWS-specific terminology.
- Core AWS services at the depth expected in a general client or developer interview.
- Networking, security, availability, scaling, storage, databases, monitoring, and cost fundamentals.
- Architecture reasoning: why you would choose one service or design over another.
- Interview checkpoints, common traps, scenario questions, and a final closed-notes recall test.

Recommended use: Read each session, then close the PDF and explain the topic aloud in your own words. Save labs and console practice for a later pass.

Session 0: How to use this guide

A full concept-only route through cloud fundamentals and AWS essentials.

Block	Sessions and purpose
Foundation	1-4: What cloud computing is, service/deployment models, virtualization, scalability, reliability, and global infrastructure.
AWS core	5-11: Accounts/IAM, networking, compute, storage, databases, messaging/API integration, and content delivery.
Operations	12-16: Observability, security, resilience/DR, pricing/cost, and deployment patterns.
Interview layer	17-20: ML/GenAI overview, architecture patterns, common questions, and closed-notes recall.

One-day concept schedule

Time block	Study target
08:00-10:00	Sessions 1-3 - cloud basics, service models, deployment models
10:15-12:15	Sessions 4-6 - global infrastructure, AWS account model, IAM
13:00-15:00	Sessions 7-8 - VPC/networking and compute
15:15-17:00	Sessions 9-10 - storage and databases
17:15-18:30	Sessions 11-13 - integration, monitoring, security
19:15-20:45	Sessions 14-16 - availability/DR, cost, deployment
21:00-22:00	Sessions 17-20 - GenAI overview, architecture, interview Q&A, recall

Depth rule

For every service, learn four things: what problem it solves, where it sits in an architecture, its main trade-off, and one realistic use case. That is more valuable than memorizing dozens of features.

Session 1: Cloud computing from first principles

Build the model that makes every AWS service easier to understand.

What is cloud computing?

Cloud computing is the on-demand delivery of computing resources - such as servers, storage, databases, networking, and software - over a network, usually the internet, with usage-based pricing and the ability to provision or release capacity quickly.

Interview-ready definition

Cloud computing lets an organization consume IT resources as elastic, on-demand services instead of purchasing, installing, and maintaining all infrastructure upfront.

On-premises vs cloud

On-premises	Cloud
Buy hardware before you need it.	Provision resources when you need them.
High capital expenditure (CapEx).	Primarily operational expenditure (OpEx).
Capacity planning is slow and conservative.	Capacity can scale up or down much faster.
You own the physical data center stack.	The provider owns and operates the underlying facilities/hardware.
Hardware refresh and maintenance are your responsibility.	Provider handles physical infrastructure lifecycle.

Five fundamental characteristics

- On-demand self-service - users can provision resources without waiting for manual hardware installation.
- Broad network access - services are reachable over standard networks and APIs.
- Resource pooling - provider infrastructure is shared safely across many customers using logical isolation.
- Rapid elasticity - capacity can expand or contract with demand.
- Measured service - usage is metered, enabling pay-for-what-you-use economics.

Benefits and trade-offs

Benefits include faster experimentation, global reach, elasticity, managed services, automation, and reduced hardware ownership. Trade-offs include recurring cost, dependency on provider services, network latency, security/configuration complexity, and possible vendor lock-in.

Common misconception

"The cloud is someone else's computer" is incomplete. The important distinction is the operating model: pooled infrastructure, APIs, automation, elasticity, managed services, and metered consumption.

Interview checkpoint

- Why would a startup prefer cloud over buying servers?
- Is cloud always cheaper? Why not?
- What is elasticity, and how is it different from simply buying a powerful server?

Session 2: Service models: IaaS, PaaS, SaaS

Understand how responsibility shifts between you and the provider.

Model	What you manage / what provider manages
IaaS - Infrastructure as a Service	You manage OS, runtime, application, and data. Provider manages physical data center, networking hardware, servers, and virtualization. Example: EC2.
PaaS - Platform as a Service	You mainly manage application code and data. Provider manages more of the OS/runtime/platform. Example style: Elastic Beanstalk / managed app platforms.
SaaS - Software as a Service	You consume a finished application. Provider manages almost the entire stack. Examples: webmail, CRM, collaboration software.

The control vs convenience spectrum

More control usually means more operational responsibility. IaaS gives flexibility but requires patching, OS hardening, capacity management, and deployment decisions. Managed platforms reduce operational work but may constrain configuration or portability.

Shared responsibility

Cloud security is shared. AWS is responsible for security OF the cloud - facilities, physical hardware, and core infrastructure. Customers are responsible for security IN the cloud - identities, permissions, data, application code, configuration, and many network controls. The exact boundary changes by service model.

Interview phrase

With EC2, the customer manages the guest OS and its patches. With a highly managed service such as S3, AWS manages much more of the underlying stack, while the customer still controls data access, policies, encryption choices, and configuration.

Interview checkpoint

- Why might a team choose PaaS over raw virtual machines?
- Who patches the operating system on an EC2 instance?
- Does moving to cloud remove the customer's security responsibility?

Session 3: Deployment models and migration thinking

Know where workloads run and why organizations mix environments.

Public cloud

Infrastructure operated by a cloud provider and delivered to many customers with logical isolation.

Private cloud

Cloud-like infrastructure dedicated to one organization, often used for regulatory, legacy, or specialized requirements.

Hybrid cloud

A connected combination of on-premises/private infrastructure and public cloud.

Multi-cloud

Using services from more than one public cloud provider. This can reduce concentration risk or meet business needs, but increases operational complexity.

Why hybrid architectures exist

- Legacy systems may be expensive or risky to migrate immediately.
- Data residency or regulatory constraints may restrict where data can live.
- Low-latency integration with factory, branch, or local systems may be required.
- Migration is often phased rather than "all at once."

Common migration strategies

Strategy	Meaning
Rehost	Move with minimal changes - often called lift and shift.
Replatform	Move while making limited platform improvements, such as adopting a managed database.
Refactor / re-architect	Redesign the application to use cloud-native patterns or managed services.
Retire	Remove workloads that are no longer needed.
Retain	Keep some workloads where they are for now.

Interview checkpoint

If asked whether every workload should be refactored immediately, the safe answer is no. Migration strategy depends on cost, risk, time, architecture, compliance, and business value.

Session 4: Scalability, elasticity, availability, and fault tolerance

These four terms appear constantly in cloud interviews and are not interchangeable.

Concept	Precise meaning
Scalability	Ability of a system to handle greater workload by adding resources.
Vertical scaling	Increase power of one machine - more CPU/RAM. Simpler, but has hard limits and may require downtime.
Horizontal scaling	Add more machines/instances. Usually better for resilient web-scale systems, but requires distributed architecture.
Elasticity	Automatically or quickly matching capacity to changing demand, including scaling back down.
High availability	Designing so the service remains available despite component failures.
Fault tolerance	Designing so the system continues operating with little or no interruption when a component fails.

Redundancy and failure domains

A reliable system avoids placing every component in one failure domain. Multiple instances, Availability Zones, replicas, and backups reduce single points of failure. Redundancy improves availability but increases cost and operational complexity.

RTO and RPO

Recovery Time Objective (RTO) is the maximum acceptable time to restore a service after a disruption. Recovery Point Objective (RPO) is the maximum acceptable amount of data loss measured in time. A lower RTO/RPO generally requires more replication, automation, and cost.

Example

If RPO is 15 minutes, the business can tolerate losing at most about 15 minutes of recent data. If RTO is 1 hour, the service should be restored within one hour of a major outage.

Session 5: AWS global infrastructure and account model

Understand the physical/logical hierarchy before learning individual services.

Regions and Availability Zones

An AWS Region is a separate geographic area. A Region contains multiple Availability Zones (AZs), which are distinct infrastructure locations designed to provide isolation from failures while remaining connected with low-latency networking.

- Choose a Region based on latency, compliance/data residency, service availability, and cost.
- A Multi-AZ design spreads critical components across failure domains.
- A Region-wide disaster is rarer, but critical systems can require cross-Region recovery strategies.

Edge locations and content delivery

Edge infrastructure places content or network entry points closer to users. Services such as CloudFront use a globally distributed edge network to reduce latency and offload origin servers.

Accounts as security and billing boundaries

An AWS account is a strong isolation, ownership, and billing boundary. Real organizations commonly use multiple accounts for production, development, security, logging, or business units, then manage them centrally with AWS Organizations.

Interview distinction

Region = geographic area. Availability Zone = isolated infrastructure location within a Region. Do not describe an AZ as simply a single building or a separate Region.

Session 6: Identity and Access Management (IAM)

Security questions often begin here. Learn identities, policies, and roles precisely.

Core IAM objects

Object	Purpose
User	Identity for a person or legacy application. Long-term access keys should be minimized.
Group	Collection of users used to apply permissions consistently.
Role	Identity with permissions that is assumed temporarily by a person, service, or workload.
Policy	JSON permission document that specifies allowed or denied actions on resources under conditions.

Authentication vs authorization

Authentication answers "Who are you?" Authorization answers "What are you allowed to do?" IAM participates in both, but permissions are fundamentally an authorization problem.

Least privilege and temporary credentials

Grant only the permissions required for a task, for only as long as needed. Prefer roles and temporary credentials for workloads instead of embedding long-lived access keys in code or configuration files.

- Protect the root user and do not use it for daily work.
- Use MFA for privileged human access.
- Deny or tightly control sensitive actions.
- Rotate or eliminate long-lived credentials where possible.

Critical interview answer

Never hard-code AWS access keys in source code. For an application running on EC2, attach an IAM role to the instance and let the AWS SDK obtain temporary credentials automatically.

Policy evaluation - simplified mental model

Start from implicit deny. An applicable allow can grant access, but an explicit deny overrides allows. In real AWS environments, several policy layers can interact, so troubleshoot permissions systematically rather than assuming one policy is the entire answer.

Session 7: VPC and networking fundamentals

You do not need to become a network engineer, but you must understand how a cloud application is connected and isolated.

Virtual Private Cloud (VPC)

A VPC is a logically isolated virtual network in AWS. You define an IP address range and divide it into subnets. Resources such as EC2 instances can be placed inside those subnets.

Public and private subnets

A subnet is considered public when its routing allows traffic to an Internet Gateway and the resource has appropriate addressing/configuration. A private subnet does not provide direct inbound internet reachability. Private workloads can still initiate outbound internet connections through a NAT device when required.

Component	Mental model
CIDR block	The IP address range assigned to the VPC or subnet.
Route table	Rules that decide where network traffic is sent.
Internet Gateway (IGW)	Connects a VPC to the public internet for appropriately configured resources.
NAT Gateway	Allows resources in private subnets to initiate outbound internet access without accepting unsolicited inbound connections.
Security Group	Stateful virtual firewall attached to supported resources. Primarily allow rules.
Network ACL	Stateless subnet-level traffic filter with allow and deny rules.
Elastic Load Balancer	Distributes traffic across multiple healthy targets.
Route 53	Managed DNS service used to map domain names to endpoints.

Security Groups vs NACLs

Security Groups are stateful: return traffic for an allowed connection is automatically permitted. Network ACLs are stateless: inbound and outbound rules are evaluated independently. In most application designs, Security Groups are the primary resource-level network control.

Architecture pattern

Internet -> Route 53 -> Load Balancer in public subnets -> application instances/services in private subnets -> database in private subnets. Security Groups allow only the required paths between tiers.

Session 8: Compute: EC2, Auto Scaling, Lambda, and containers

Know the major compute choices and when each is appropriate.

Amazon EC2

EC2 provides resizable virtual machines. You choose an instance family/size, operating system image, network placement, storage, and security configuration. EC2 is appropriate when you need OS-level control or a traditional server environment.

- AMI - template used to launch an instance.
- Instance type - determines compute, memory, networking, and other capacity characteristics.
- EBS - common persistent block storage attached to EC2.
- User data - initialization script/data that can run during launch.

Auto Scaling and load balancing

An Auto Scaling Group maintains a desired number of instances and can adjust capacity using metrics or schedules. A load balancer distributes requests across healthy targets. Together they enable horizontal scaling and improve availability.

AWS Lambda

Lambda runs code in response to events without the customer managing servers. It is useful for event-driven processing, APIs, automation, lightweight backends, and workloads with variable demand. You still manage code, dependencies, permissions, observability, and architecture.

Containers: ECS and EKS

Containers package an application and its dependencies into portable runtime units. ECS is AWS's container orchestration service; EKS provides managed Kubernetes control planes. Containers are useful when you want consistent packaging, service isolation, and orchestration without managing each application as a full VM.

Need	Likely starting point
Full OS control / legacy server	EC2
Short event-driven function	Lambda
Containerized service with AWS-native orchestration	ECS
Kubernetes ecosystem / portability requirements	EKS

Interview trade-off

Serverless does not mean "no servers exist." It means the provider manages server provisioning and much of the operational layer for you.

Session 9: Storage: S3, EBS, EFS, and archival tiers

Choose storage based on the access model, not just capacity.

Service	Best mental model
Amazon S3	Highly scalable object storage accessed using object keys/APIs. Great for files, backups, data lakes, static assets, logs, and media.
Amazon EBS	Block storage volume typically attached to EC2. Think virtual disk for a server.
Amazon EFS	Managed shared file system that multiple Linux-based compute instances can mount concurrently.
S3 archival classes	Lower-cost storage for infrequently accessed or archival data, with different retrieval characteristics.

S3 fundamentals

- Bucket - top-level container with globally unique naming requirements within AWS partition rules.
- Object - data plus metadata, identified by a key.
- Versioning - retains multiple object versions and helps recover from accidental overwrite/deletion.
- Lifecycle rules - transition or expire objects automatically based on age or policy.
- Encryption - data can be encrypted at rest and in transit; key-management choices affect control and operations.

Object vs block vs file storage

Object storage is API/key based and scales extremely well. Block storage exposes raw volumes to a host and is suited to operating systems and databases. File storage provides familiar directory/file semantics and shared mounting.

Common interview trap

You normally do not attach S3 to EC2 as if it were an EBS disk. They expose different storage models and are chosen for different access patterns.

Session 10: Databases: RDS/Aurora, DynamoDB, and caching

Start with data model, consistency, access pattern, and operational needs.

Relational databases

Amazon RDS manages common relational database engines. It automates much of the infrastructure work such as provisioning, backups, patching workflows, and high-availability options while you still design schemas, indexes, queries, access controls, and application logic.

Amazon Aurora is an AWS-designed relational database compatible with MySQL/PostgreSQL ecosystems, built around a distributed managed storage architecture.

DynamoDB

DynamoDB is a managed NoSQL key-value/document database designed for very low-latency access at scale. It works best when access patterns and partition keys are designed deliberately. It is not simply a faster replacement for every relational database.

Caching

ElastiCache provides managed in-memory caching engines. Caches reduce repeated database work and latency but introduce invalidation, consistency, and memory-management concerns.

Question	Direction
Complex joins, transactions, relational schema?	RDS/Aurora is often the natural starting point.
Massive key-based access with predictable patterns?	DynamoDB may be a strong fit.
Repeated reads need microsecond/millisecond acceleration?	Add a cache such as ElastiCache where justified.

Interview rule

Do not choose a database because it is "popular." Start from access patterns, consistency/transaction needs, scaling pattern, latency target, and operational constraints.

Session 11: APIs, messaging, events, and content delivery

Distributed systems become easier when components are decoupled.

API Gateway

API Gateway provides managed HTTP/WebSocket API front doors that can route requests to Lambda, services, or other backends. It can centralize concerns such as routing, authentication integration, throttling, and API management.

SQS, SNS, and EventBridge

Service	Use it when...
SQS	You need a durable queue that decouples producers from consumers and smooths workload spikes.
SNS	You need publish/subscribe fan-out so one message can be delivered to multiple subscribers.
EventBridge	You need event routing between applications/services using rules and event patterns.

Why asynchronous messaging matters

If every component requires an immediate synchronous response from every other component, failures and traffic spikes propagate easily. Queues let producers hand off work and allow consumers to process it at their own rate, improving resilience and burst handling.

CloudFront

CloudFront is a content delivery network (CDN). It caches or serves content from edge locations closer to users, reducing latency and origin load. It is often placed in front of S3, load balancers, or APIs depending on the use case.

Scenario

Image upload -> S3 -> event -> queue -> worker/Lambda processes image -> result stored -> notification/event emitted. This avoids forcing the user's request to wait for all processing synchronously.

Session 12: Monitoring, logging, and auditability

A production system is not complete unless you can understand its behavior.

CloudWatch

CloudWatch is the central AWS service family for metrics, logs, alarms, and operational visibility. You use it to observe resource health, application behavior, and thresholds that may require action or automated scaling.

CloudTrail

CloudTrail records many AWS API/account activity events. It is primarily an audit trail: who or what made an API call, what was changed, and when. It is different from application logging.

AWS Config

AWS Config records and evaluates resource configuration history and can help identify drift or non-compliant configurations.

Need	Primary concept
CPU/latency/error-rate metrics	CloudWatch metrics
Application/server logs	CloudWatch Logs
Alert when threshold is crossed	CloudWatch Alarm
Who changed an AWS resource?	CloudTrail
How has a resource configuration changed?	AWS Config

Observability signals

- Metrics - numerical time series such as latency, CPU, request count, queue depth.
- Logs - detailed event records useful for diagnosis and auditing.
- Traces - follow a request across distributed components to understand latency and failures.

Interview answer

Monitoring tells you that something is wrong. Good observability helps you understand why it is wrong using metrics, logs, traces, and contextual data.

Session 13: Cloud security fundamentals

Security is a design property, not a single AWS service.

Defense in depth

Use multiple independent controls: strong identity, least privilege, network segmentation, encryption, application validation, logging, backups, patching, and threat detection. Do not rely on one firewall or one IAM policy as the entire security strategy.

Encryption

- In transit - protect network traffic, commonly with TLS/HTTPS.
- At rest - encrypt stored data using service-managed or customer-controlled keys depending on requirements.
- KMS - managed key-management service used by many AWS services for encryption keys and access control integration.

Secrets

Secrets such as database passwords, API credentials, and tokens should not be embedded in source code or images. Managed secret/configuration systems let applications retrieve sensitive values securely with IAM-controlled access and rotation workflows.

Network/application protection

- Security Groups and subnet design restrict network paths.
- AWS WAF can filter malicious or unwanted web requests at supported entry points.
- AWS Shield provides DDoS protection capabilities, with baseline protections integrated into AWS and advanced options for higher requirements.

Secure default

Private database subnet + no public IP + Security Group accepts database traffic only from the application tier
+ encrypted storage + secrets stored outside source code + monitoring/auditing enabled.

Session 14: High availability, backups, and disaster recovery

Learn to design for failure instead of assuming components never fail.

High availability patterns

- Run multiple application instances instead of one critical server.
- Spread resources across multiple Availability Zones when the service supports it.
- Use health checks and load balancing so failed targets stop receiving traffic.
- Use managed database Multi-AZ/replication options where appropriate.
- Automate instance replacement and scaling rather than relying on manual recovery.

Backups are not the same as high availability

High availability reduces interruption during failures. Backups let you restore data or systems after loss/corruption. A system can be highly available and still propagate bad data; backups provide a separate recovery mechanism.

Disaster recovery strategies - concept only

Pattern	Trade-off
Backup and restore	Lowest ongoing cost, but generally higher recovery time.
Pilot light	Keep critical core components ready; scale the rest after disaster.
Warm standby	Run a smaller but functional copy that can scale up.
Active/active multi-site	Multiple sites actively serve traffic; strongest continuity but highest complexity/cost.

Interview framing

Pick DR based on business RTO/RPO. There is no universal "best" recovery architecture because tighter objectives cost more.

Session 15: Cloud economics and cost fundamentals

Understand why "pay as you go" does not mean "cheap by default."

CapEx vs OpEx

Traditional data centers often require significant upfront capital expenditure. Cloud shifts much spending toward operational expenditure, allowing organizations to consume resources as needed. This improves agility but requires active cost governance.

Common cost drivers

- Compute runtime and instance size.
- Storage capacity and storage class.
- Database size, throughput, replicas, and backups.
- Data transfer - especially between locations or out to the internet.
- Managed-service requests, processing, and specialized features.

EC2 pricing concepts

Option	Concept
On-Demand	Flexible, pay for usage without long commitment. Good baseline for unpredictable workloads.
Savings Plans / Reserved-style commitments	Lower effective pricing in exchange for longer-term usage commitment.
Spot	Uses spare capacity at deep discounts but can be interrupted; ideal for fault-tolerant workloads.

Cost control habits

- Right-size resources instead of assuming bigger is safer.
- Scale down or stop non-production environments when idle.
- Use lifecycle policies for old data.
- Set budgets/alerts and tag resources for ownership and cost allocation.
- Measure unit economics, not only the monthly bill.

Interview answer

Cloud cost optimization is continuous. The cheapest architecture is not always the best if it sacrifices reliability, developer productivity, or performance.

Session 16: Application deployment: Elastic Beanstalk and the managed-platform mindset

Connect AWS fundamentals to a common developer deployment workflow.

What Elastic Beanstalk does

Elastic Beanstalk is a managed application deployment service. You provide application code/package and configuration; it provisions and coordinates underlying AWS resources such as compute, load balancing, scaling, and monitoring according to the environment setup.

Why use it?

- Faster deployment than manually wiring every EC2 component yourself.
- Retains access to underlying AWS resources while automating common platform operations.
- Useful for conventional web applications when you want a managed deployment experience without redesigning everything around serverless or containers.

Typical web request path

Request flow

Client -> DNS -> Load Balancer -> application instance(s) -> application runtime (for example Flask/Gunicorn) -> database/API/storage -> response

Interview questions to prepare

- Why did you use Elastic Beanstalk instead of manually managing EC2?
- What is the purpose of a production WSGI server such as Gunicorn for a Python web app?
- Where should environment secrets and API keys be stored?
- How would you scale the application if traffic suddenly increases?
- How would you monitor failures and latency?
- What would you change to make the deployment highly available?

Strong framing

Describe the trade-off: Beanstalk reduces infrastructure setup effort while still using familiar AWS building blocks. It is a platform choice, not magic.

Session 17: AWS for ML and Generative AI - only the fundamentals

Know the service landscape without turning this week into an AWS ML certification course.

SageMaker

Amazon SageMaker is a managed ML platform covering stages such as data preparation, model development, training, deployment, and MLOps. In an interview, know that it reduces the need to build the entire ML infrastructure stack yourself.

Amazon Bedrock

Amazon Bedrock is a managed service for building generative AI applications using foundation models and related capabilities. At the fundamentals level, think model access + application integration + managed GenAI tooling rather than managing model-serving infrastructure from scratch.

Supporting services

- S3 commonly stores datasets, model artifacts, documents, and logs.
- Lambda or container/compute services can host preprocessing, API orchestration, or event-driven components.
- IAM controls which workloads can access models and data.
- CloudWatch provides operational visibility.
- Databases/vector-capable data stores may support application state or retrieval workflows depending on architecture.

Interview scope

For a general client interview, it is enough to explain where SageMaker or Bedrock fits. Do not pretend to know every feature if you have not used it.

Session 18: Core AWS architecture patterns

Use these as mental templates when an interviewer gives you a scenario.

Pattern A - Highly available web application

Flow

Route 53 -> Load Balancer -> app instances/tasks across 2+ AZs -> managed database with HA -> S3 for static/object data -> CloudWatch for monitoring

Key reasoning: remove single points of failure, scale horizontally, isolate database access, and monitor the entire path.

Pattern B - Serverless API

Flow

Client -> API Gateway -> Lambda -> DynamoDB/S3 -> CloudWatch

Key reasoning: useful for event-driven or variable workloads where server management should be minimized. Watch for execution/runtime limits, cold-start sensitivity, downstream bottlenecks, and cost at sustained high volume.

Pattern C - Asynchronous processing

Flow

API/app -> SQS -> workers/Lambda -> database/S3 -> SNS/EventBridge notification

Key reasoning: queue absorbs spikes and isolates producer availability from consumer processing speed.

Pattern D - Static website

Flow

Route 53 -> CloudFront -> S3

Key reasoning: object storage hosts static assets while CDN edge caching improves latency and reduces origin load.

Pattern E - Private three-tier application

Flow

Public load balancer -> private app tier -> private database tier; NAT only for outbound needs; tightly scoped Security Groups

Key reasoning: expose only the entry point that must be public. Keep internal compute/data tiers private unless there is a specific need.

Session 19: High-frequency interview questions and model answers

Use these as answer frameworks, not scripts to memorize word-for-word.

What is cloud computing?

On-demand access to pooled IT resources delivered as services, with rapid provisioning, elasticity, and metered usage. It changes both the technology and the operating/economic model.

Scalability vs elasticity?

Scalability is the ability to handle increased workload by adding resources. Elasticity is the dynamic ability to match capacity to changing demand, including scaling back down.

Region vs Availability Zone?

A Region is a geographic area; an AZ is an isolated infrastructure location within a Region. Multi-AZ architecture reduces dependence on one failure domain.

EC2 vs Lambda?

EC2 gives server/OS control and is suited to long-running or custom environments. Lambda removes server provisioning and fits event-driven functions, but introduces runtime/event model constraints.

S3 vs EBS?

S3 is object storage accessed by API/object key. EBS is block storage that behaves like a disk attached to compute.

Security Group vs NACL?

Security Groups are stateful and resource-level. NACLs are stateless and subnet-level, with explicit inbound/outbound evaluation.

RDS vs DynamoDB?

RDS is relational and fits SQL/transactional schemas. DynamoDB is managed NoSQL optimized for known key-based access patterns and large-scale low-latency workloads.

Why use a load balancer?

To distribute traffic across multiple healthy targets, avoid dependence on one server, improve availability, and support horizontal scaling.

Why use a queue?

To decouple producers and consumers, absorb bursts, and prevent slow downstream processing from blocking the request path.

What is shared responsibility?

AWS secures the cloud infrastructure; customers secure their identities, data, applications, permissions, and configurations. The exact boundary depends on the service.

Session 20: Common traps and closed-notes recall

This is the final check before you move to labs or mock interviews.

Common traps

- Cloud is not automatically cheaper. Idle or poorly designed resources can be expensive.
- "Serverless" does not mean there are no servers; it means you do not manage the server infrastructure directly.
- A public subnet does not automatically make every resource publicly reachable. Routing, addressing, and security controls also matter.
- Backups do not by themselves provide high availability.
- Multi-AZ and multi-Region solve different levels of failure and cost very different amounts.
- DynamoDB is not simply "better RDS." Database choice depends on data model and access patterns.
- S3, EBS, and EFS are not interchangeable because they expose object, block, and file storage models.
- IAM roles are generally preferred over hard-coded long-term access keys for workloads.
- Encryption does not replace authorization. Encrypted data can still be exposed by bad permissions.
- Auto Scaling cannot fix an application that stores all critical state locally on one instance.

28-question closed-notes recall

1. Define cloud computing in your own words.
2. List the five core cloud characteristics.
3. What is the difference between CapEx and OpEx?
4. Explain IaaS, PaaS, and SaaS.
5. What does shared responsibility mean?
6. Public cloud vs private cloud vs hybrid cloud?
7. Scalability vs elasticity?
8. Vertical vs horizontal scaling?
9. High availability vs fault tolerance?
10. Define RTO and RPO.
11. Region vs Availability Zone?
12. What is an IAM role and why are temporary credentials useful?
13. Authentication vs authorization?
14. What is a VPC?
15. Public vs private subnet?
16. Internet Gateway vs NAT Gateway?
17. Security Group vs Network ACL?
18. What does a load balancer do?
19. What is Auto Scaling?
20. EC2 vs Lambda?
21. S3 vs EBS vs EFS?
22. RDS/Aurora vs DynamoDB?
23. Why add a cache?
24. SQS vs SNS?
25. CloudWatch vs CloudTrail?
26. What are three layers of defense in depth?
27. Why can cloud costs grow unexpectedly?
28. Describe one highly available web application architecture on AWS.

Session A: Final interview cheat sheet

The shortest version of the entire guide.

Term	One-line answer
Cloud computing	On-demand, elastic, metered IT resources delivered as services.
IaaS / PaaS / SaaS	Increasing levels of provider management; decreasing customer operational responsibility.
Region / AZ	Geographic area / isolated failure domain within that Region.
IAM	Identity, authentication/authorization, roles, and policies.
VPC	Logically isolated virtual network in AWS.
Security Group	Stateful resource firewall.
NACL	Stateless subnet-level network filter.
EC2	Virtual machines.
Lambda	Event-driven serverless compute.
S3	Object storage.
EBS	Block storage for compute.
EFS	Managed shared file storage.
RDS/Aurora	Managed relational databases.
DynamoDB	Managed NoSQL key-value/document database.
ELB	Traffic distribution across healthy targets.
Auto Scaling	Automatically maintain/adjust compute capacity.
SQS	Durable queue for decoupling.
SNS	Publish/subscribe fan-out.
CloudFront	CDN/edge delivery.
Route 53	DNS.
CloudWatch	Metrics, logs, alarms, operational visibility.
CloudTrail	Audit trail of AWS API/account activity.
KMS	Managed encryption key service.
RTO / RPO	Time to recover / acceptable data-loss window.

The architecture answer formula

When given a scenario, answer in this order: requirements -> traffic/data characteristics -> availability target -> security boundary -> service choices -> scaling -> observability -> failure handling -> cost trade-offs. Interviewers care more about your reasoning than about naming the maximum number of AWS services.

Final target

If you can explain every row on this page clearly without looking at the PDF, you have the AWS/cloud fundamentals needed for a general developer/client interview. Then move to hands-on console practice and scenario drills.